

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 641 — Computer Graphics

The real-time pipeline from first principles: transforms, rasterization, shading, texturing, and animation

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 1 (with CSCE 629 and CSCE 614)

Prerequisites: Linear algebra (UC LINA 600); C++ fluency; no prior graphics assumed

Effort: 10–12 hours per week · 13 modules + 2 project weeks

Deliverable: A software rasterizer written from scratch, then the same scene reproduced on the GPU with a programmable pipeline

Course description

Computer graphics is the problem of deciding, for every pixel on a screen, what colour it should be — fast enough that the answer arrives sixty times a second. Stated that way it sounds like a question about colour. It is really a question about geometry, sampling, and parallelism, and the colour falls out at the end.

This course builds the real-time pipeline from the bottom. You will write a rasterizer that has no GPU underneath it, so that nothing is magic: your own matrix stack, your own triangle setup, your own depth test, your own texture filtering. Only once that works do we move to the GPU, where the same stages reappear as hardware and the parts you programmed by hand become the parts you configure.

The emphasis throughout is on *why each stage exists*. Almost every stage in the pipeline is there to make a sampling problem tractable or to exploit parallelism, and once you can see which, the design stops looking arbitrary. By the end you should be able to look at a rendering artifact and localise it to a stage before you open a debugger.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Derive and compose the model, view, and projection transforms, and explain what the perspective divide does geometrically.
- 2 Implement triangle rasterization with correct fill rules, perspective-correct interpolation, and depth testing.
- 3 Explain aliasing as a sampling phenomenon and implement at least two antialiasing strategies with their trade-offs.

- 4 Write vertex and fragment shaders for a physically based material under image-based lighting.
- 5 Implement shadow mapping and diagnose its characteristic artifacts.
- 6 Build a transform hierarchy with skinned skeletal animation.
- 7 Localise a rendering bug to a pipeline stage from the artifact alone.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	The Graphics Pipeline, End to End	What actually happens between a vertex buffer and a lit pixel?
2	Vectors, Frames, and Homogeneous Coordinates	Why does a 3D renderer use 4D coordinates?
3	Transformations: Model, View, Projection	Where does each of the three matrices come from, and why that order?
4	Rasterization: From Triangles to Fragments	How do you decide coverage without gaps, overlaps, or swimming textures?
5	Visibility: Depth, Culling, and the Z-Buffer	How do you resolve occlusion without sorting the whole scene?
6	Light, Colour, and Local Shading	What determines the colour of a point on a surface?
7	The Programmable Pipeline: Shaders in Practice	How does the pipeline you wrote by hand map onto real hardware?
8	Texture Mapping, Filtering, and Mipmaps	How do you map an image onto a surface without shimmering?
9	Sampling, Aliasing, and Antialiasing	Why do jagged edges, shimmering textures, and lost highlights all have the same cause?
10	Shadows	How do you decide whether a point can see the light?
11	Physically Based Shading and Image-Based Lighting	What would a shading model look like if it were derived rather than fitted?
12	Geometry: Meshes, Curves, and Tessellation	What is the right data structure for a surface?
13	Animation: Hierarchies, Skinning, and Interpolation	How do you deform a mesh smoothly with a skeleton?

Projects

Project 1 — Software rasterizer (after Module 5)

Write a complete software rasterizer: load a triangle mesh, transform it through the full MVP chain, rasterize with correct fill rules and perspective-correct interpolation, depth-test it, and write a PNG. No GPU, no graphics API, no matrix library you did not write.

Requirements

- Loads an OBJ mesh with positions, normals, and texture coordinates.
- Full model/view/projection chain with a configurable camera, including perspective divide and viewport transform.
- Triangle rasterization using edge functions, with a consistent fill rule so that shared edges are neither doubled nor gapped.
- Perspective-correct interpolation of all vertex attributes.
- Z-buffer with correct depth precision handling.
- Backface culling and near-plane clipping.

Done means

- Renders a 5,000-triangle mesh correctly at 1920×1080.
- A screenshot showing no cracks along shared triangle edges under magnification.
- A second screenshot with perspective-correct interpolation disabled, demonstrating that you can see the difference and explain it.

Project 2 — GPU renderer with physically based shading (after Module 11)

Reproduce the same scene on the GPU with a programmable pipeline, then take it past what the software renderer could do: a physically based material model under image-based lighting, with shadow mapping and antialiasing.

Requirements

- Vertex and fragment shaders implementing a Cook–Torrance microfacet BRDF with metallic/roughness parameters.
- Image-based lighting from an HDR environment map, with both diffuse irradiance and specular prefiltering.
- Shadow mapping with a documented strategy for acne and peter-panning.
- At least one antialiasing method, with a measured frame-cost comparison against no antialiasing.
- A RenderDoc capture of one frame, annotated with what each draw call contributes.

Done means

- A material sphere grid sweeping roughness against metallic, rendered under a real HDR environment.
- Side-by-side shadow comparison showing the artifact before your fix and after it.
- A written paragraph on where the frame time actually goes, backed by the capture rather than by guesswork.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
GAMES101 — Lingqi Yan, Introduction to Modern Computer Graphics (free, English slides)	Primary lecture series. Clear, modern, and closely aligned to this course's order. Watch the mapped lecture before each module.
CMU 15-462/662 Computer Graphics — Keenan Crane (free video)	Second voice, more mathematical. Best for geometry, sampling, and the parts where you want the theory tightened.
Cem Yuksel — Interactive Computer Graphics, Utah (free video)	Best practical treatment of the GPU pipeline and the API layer. Use from Module 07 onward.
UC San Diego CSE167x Computer Graphics (edX, free audit)	Structured assignments with automated feedback. Good if you want graded checkpoints.
Scratchapixel — graphics from scratch (free)	Written reference with full derivations. The place to go when a lecture moved too fast.
LearnOpenGL (free)	The API tutorial. Use for mechanics only — the theory comes from this course, not from here.

Tooling

- **C++17 or newer**, with CMake. The rasterizer should have no dependencies beyond a PNG writer — that is the point.
- **GLM** for vector and matrix types once you are past Module 03. Write your own first, then switch, so you know what it does.
- **OpenGL 4.5 core** or **Vulkan** from Module 07. OpenGL is recommended for this course: less ceremony per concept.
- **RenderDoc** for frame capture. Non-negotiable from Module 07 — reading a captured frame is a core skill, not a debugging last resort.
- **A portfolio repository**, public, from day one. Each module's output committed with a README image.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.